PRD — Pipeline agéntico de prospección de proyectos de marketing digital

4 nov 2026 · @Someone

Resumen

El pipeline detecta proyectos de UX/UI y SEO publicados en plataformas freelance, empezando por Workana, los puntúa y estima su rentabilidad para decidir a cuáles postularse. En fases posteriores produce la guía de estilo, el contenido y la web del cliente.

La fase 1 se construye en Claude Code con agentes escritos en prosa, se ejecuta desde la consola y se valida con un dashboard que muestra si el scraping y el análisis son correctos.

- **Para quién:** el equipo que hoy busca proyectos a mano y decide dónde postularse.
- **Problema:** encontrar oportunidades rentables lleva horas, la calidad de los posts es muy desigual y no hay datos propios de precios de mercado.
- **Resultado de la fase 1:** un listado actualizado de proyectos relevantes, normalizados, deduplicados y con rating, más una vista para auditar cada extracción contra la página original.

Objetivos y alcance

La fase 1 se considera lograda cuando el scraping de Workana es confiable y medible contra un golden set; el análisis profundo y la generación quedan para después. Las metas son iniciales y se recalibran con las primeras corridas.

Objetivo	Métrica	Meta inicial	Cómo se mide
Capturar todo lo relevante	Recall de captura por corrida	⩾ 95%	Conteo manual de la búsqueda vs. proyectos guardados
Extraer bien los datos	Exactitud por campo (título, presupuesto, categoría, fecha, propuestas)	⩾ 95%	Golden set etiquetado a mano
Quedarse con lo que interesa	Precisión / recall del filtro de relevancia	⩾ 90% / ⩾ 85%	Golden set etiquetado a mano
No repetir proyectos	Tasa de duplicados	< 2%	Clave canónica por fuente + id externo
Priorizar como lo	Correlación del rating con	Spearman ⩾ 0,6	30 proyectos ordenados

haría el equipo	el ranking humano		por dos personas
Controlar el gasto	Costo LLM por proyecto procesado	Medir en fase 0, fijar tope en fase 1	Log de tokens por agente y corrida

Dentro del alcance de las fases iniciales

- Registro configurable de fuentes y criterios de búsqueda (empezando por Workana: UX/UI y SEO).
- Scraping, normalización, deduplicación y almacenamiento con snapshot del HTML original.
- Filtro de relevancia, rating de fuentes y rating de calidad de cada post.
- Análisis de brief, estimación de rentabilidad y tendencias de precios con datos propios.
- Dashboard de validación y ejecución completa desde la consola.

Fuera del alcance de las fases iniciales

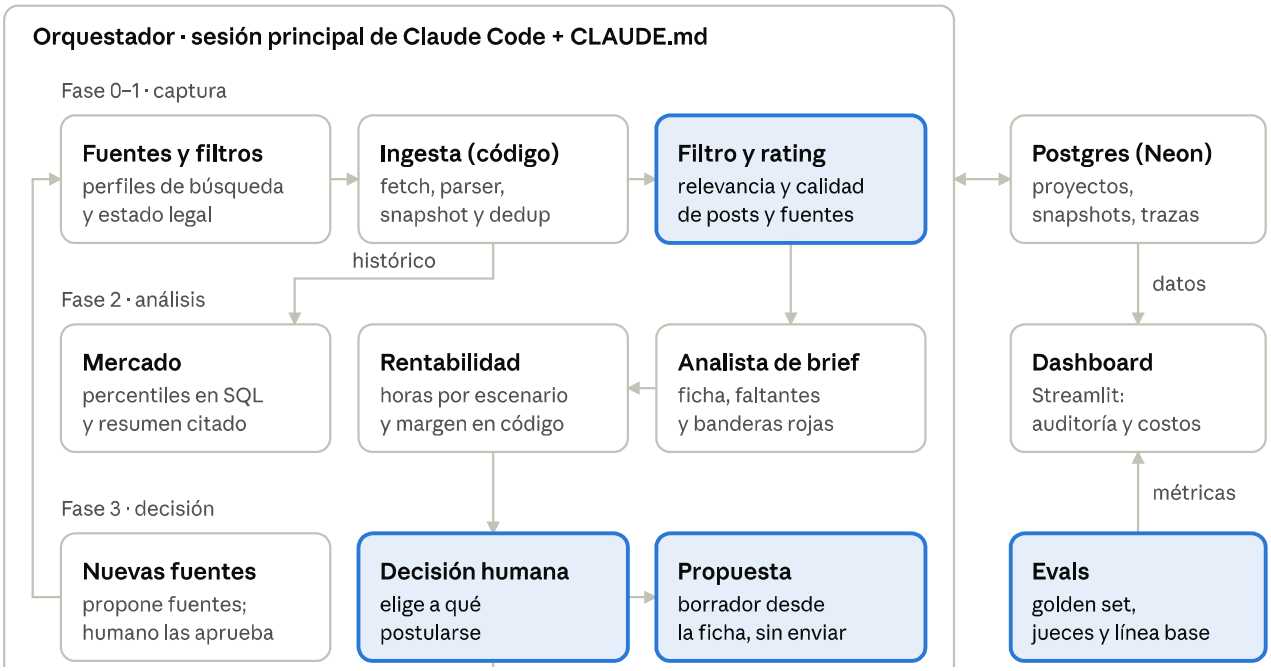
- Enviar propuestas o mensajes automáticamente en nombre de la cuenta.
- Iniciar sesión con credenciales personales en las plataformas.
- Generación de contenido, UX/UI y construcción de webs (fases 4 y 5).
- Multiusuario, autenticación y migración a LangGraph.

Arquitectura revisada

El draft se mantiene en espíritu, con tres cambios de fondo: la ingesta pasa a ser código, aparecen el rating y una decisión humana antes de producir, y la base, el dashboard y los evals existen desde la fase 1.

La ingesta es código; los agentes juzgan y redactan sobre una base común

Pipeline revisado: la sesión principal llama a cada etapa en orden



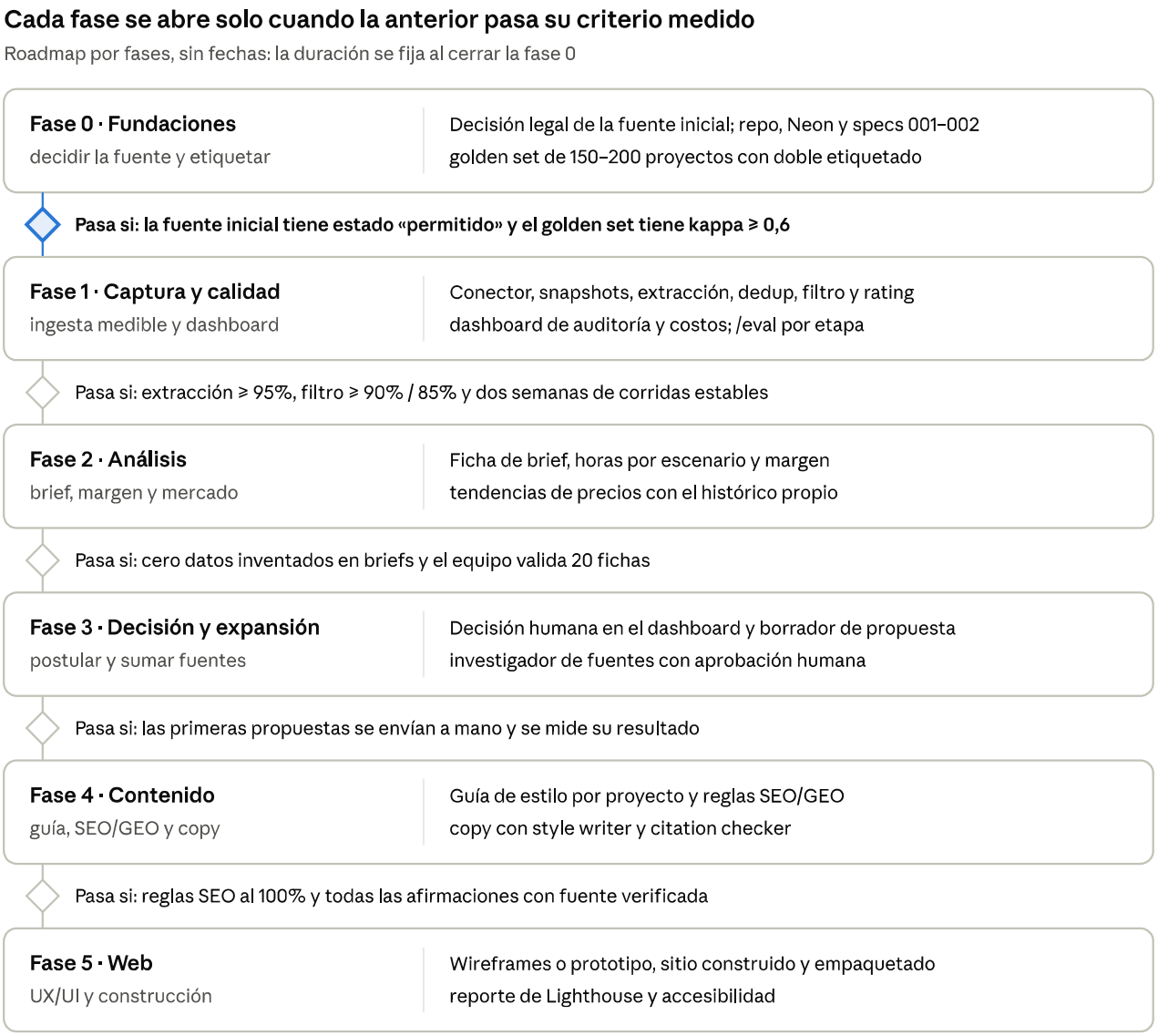


La ejecución baja fase por fase; solo los proyectos que una persona elige pasan a propuesta, contenido y web.

Cambio respecto del draft	Por qué
La ingesta es un script, no un agente	Es determinista, barata y testeable; el LLM solo interpreta texto
«Sitios» y «gold-standard» pasan a registro y golden set versionados	Son configuración y datos de evaluación, no agentes
Nueva etapa de filtro y rating	El rating de sitios y posts no estaba en el draft y es lo que permite priorizar
Brief y rentabilidad separados	Interpretar texto (LLM) y calcular margen (código) se evalúan distinto
Decisión humana antes de producir	Contenido y web son caros; solo se producen para proyectos elegidos
Nueva etapa de propuesta	Convierte un proyecto encontrado en una postulación; queda como borrador
El investigador alimenta el registro de fuentes	Una fuente nueva entra con estado legal y aprobación, no directo al scraper
Base y dashboard desde la fase 1	El dashboard del draft estaba al final; para validar el scraping hace falta al principio
Evals como componente propio	Cada etapa se mide antes de que otra dependa de ella
Orquestación plana	Más simple de depurar; los subniveles del draft pasan a etapas hermanas o skills

Roadmap por fases

La fase 0 bloquea todo lo demás: sin una fuente con acceso permitido y un golden set confiable no hay nada que medir. Las fases se encadenan por criterios de salida, no por fechas.



La migración a LangGraph se evalúa cuando captura y análisis den resultados estables; los agentes en prosa y las salidas con esquema se trasladan como nodos.

Requerimientos funcionales

Cada requerimiento está en formato EARS para pasarlo directo a las specs SDD; la fase indica cuándo entra. Todo cálculo numérico (presupuestos, márgenes, tendencias) se hace en código; el LLM interpreta texto y redacta, nunca calcula.

Registro de fuentes y criterios (fase 0)

- **RF-FUE-01** — El sistema debe mantener un registro de fuentes con nombre, URL base, modo de acceso (API, feed, email, scraping, manual), estado legal (permitido, pendiente, prohibido), frecuencia y límite de tasa.
- **RF-FUE-02** — El sistema debe permitir definir búsquedas por fuente: categorías,

palabras incluidas y excluidas, idiomas, presupuesto mínimo y antigüedad máxima. Ejemplo inicial: Workana, UX/UI y SEO.

- **RF-FUE-03** — Si el estado legal de una fuente no es «permitido», entonces el sistema no debe ejecutar su conector automático.
- **RF-FUE-04** — El sistema debe versionar los criterios y registrar en cada corrida qué versión usó.

Ingesta (fase 1)

- **RF-ING-01** — Cuando se ejecuta la ingesta de una fuente, el sistema debe obtener los listados, guardar el snapshot crudo (HTML o JSON) y registrar la corrida: inicio, fin, páginas, ítems y errores.
- **RF-ING-02** — El conector debe respetar robots.txt, los términos de la fuente y el límite de tasa configurado.
- **RF-ING-03** — Si la fuente responde con CAPTCHA o bloqueo anti-bot, entonces el conector debe detenerse, marcar la corrida como bloqueada y no intentar evadirlo.
- **RF-ING-04** — El sistema debe extraer los campos estructurados con parsers deterministas; el LLM solo completa campos de texto libre.
- **RF-ING-05** — Si en una corrida más del 10% de los ítems llega sin un campo obligatorio, entonces el sistema debe marcar «cambio de estructura» y no pasar esa corrida al análisis.
- **RF-ING-06** — El sistema debe deduplicar por fuente + id externo y marcar casi-duplicados entre fuentes por similitud de título y descripción.
- **RF-ING-07** — Cuando un proyecto ya guardado cambia de presupuesto, cantidad de propuestas o estado, el sistema debe guardar una nueva versión, no sobrescribir.

Filtro y clasificación (fase 1)

- **RF-CLA-01** — Para cada proyecto nuevo, el sistema debe asignar categoría (UX/UI, SEO, ambas, otra), subcategorías, idioma y una relevancia de 0 a 1 con justificación de una línea.
- **RF-CLA-02** — El sistema debe aplicar primero reglas de palabras clave y enviar al LLM solo los casos ambiguos.
- **RF-CLA-03** — Si la salida del LLM no valida contra el esquema JSON, entonces el sistema debe reintentar una vez y, si vuelve a fallar, marcar el proyecto como «a revisar».

Rating de posts y de fuentes (fase 1)

- **RF-RAT-01** — El sistema debe puntuar la calidad de cada post de 0 a 100 con una rúbrica: claridad del alcance, presupuesto explícito, entregables, plazos, señales del cliente (pago verificado, historial) y señales de riesgo (trabajo gratis de prueba, contacto fuera de la plataforma).
- **RF-RAT-02** — El sistema debe calcular el rating de cada fuente: porcentaje de posts

RF-RAT-02 — El sistema debe calcular el rating de cada fuente: porcentaje de posts relevantes, calidad media, presupuesto mediano, volumen semanal, tasa de errores de ingesta y estado legal.

- **RF-RAT-03** — La rúbrica debe vivir en un archivo versionado y cada rating debe guardar la versión usada.
- **RF-RAT-04** — Cuando una persona corrige una clasificación o un rating, el sistema debe guardar la corrección como etiqueta reutilizable en el golden set.

Análisis de brief y rentabilidad (fase 2)

- **RF-BRF-01** — Para cada proyecto con relevancia ⩾ umbral, el sistema debe producir una ficha: objetivo de negocio, entregables, plataforma o stack, tamaño (páginas, pantallas), restricciones, información faltante y banderas rojas.
- **RF-BRF-02** — El sistema debe generar las preguntas que habría que hacerle al cliente antes de cotizar.
- **RF-REN-01** — El sistema debe estimar horas por tarea en tres escenarios (bajo, medio, alto) usando una tabla de referencia del equipo, no la intuición del modelo.
- **RF-REN-02** — El sistema debe calcular en código: costo = horas × tarifa interna; margen = presupuesto − costo − comisión de la plataforma; y la variación del margen entre escenarios.
- **RF-REN-03** — El sistema debe estimar la competencia (propuestas recibidas, antigüedad) y combinarla con el margen en un puntaje de prioridad.
- **RF-REN-04** — La comisión y la tarifa interna deben ser parámetros por fuente y por equipo, no valores fijos en prompts.

Tendencias de mercado (fase 2)

- **RF-MER-01** — El sistema debe calcular con el histórico propio la mediana y los percentiles 25 y 75 de presupuesto por categoría y semana, el volumen y la competencia media.
- **RF-MER-02** — Cuando una métrica cambia más de un umbral configurable contra las 4 semanas previas, el sistema debe registrarla como señal de tendencia.
- **RF-MER-03** — El resumen de tendencias redactado por el agente solo puede usar números devueltos por consultas SQL, citando la consulta.

Descubrimiento de fuentes y propuesta (fase 3)

- **RF-INV-01** — El investigador debe proponer fuentes candidatas con URL, tipo de proyectos, volumen estimado, modo de acceso disponible y el enlace a sus términos de uso.
- **RF-INV-02** — El sistema no debe activar una fuente nueva sin aprobación humana explícita.
- **RF-PRO-01** — Donde el equipo decida postularse, el sistema debe redactar un borrador de propuesta a partir de la ficha del brief; nunca debe enviarla.

Contenido (fase 4)

- **RF-CON-01** — El sistema debe generar una guía de estilo por proyecto: voz, tono, paleta y tipografía sugeridas, y referencias citadas.
- **RF-CON-02** — El sistema debe generar reglas SEO/GEO: palabras clave, arquitectura de información, metadatos, datos estructurados y bloques citables por motores generativos.
- **RF-CON-03** — El copywriter debe redactar por página siguiendo la guía; el style writer ajusta la voz y el citation checker valida que cada afirmación factual tenga fuente y que el texto sea original.

UX/UI y construcción web (fase 5)

- **RF-WEB-01** — El sistema debe producir wireframes o un prototipo de las pantallas principales a partir del brief y la guía de estilo.
- **RF-WEB-02** — El constructor debe generar el sitio, empaquetarlo (repositorio, build y vista previa) y adjuntar el reporte de calidad (accesibilidad, rendimiento, SEO técnico).

Modelo de datos

Postgres en Neon con 13 tablas; todo resultado de un agente guarda modelo, versión de prompt o rúbrica y corrida, para poder comparar versiones en los evals. No se guardan nombres ni contactos de los clientes que publican, solo señales agregadas (país, pago verificado, historial).

Entidad	Campos clave	Para qué
`sources`	nombre, url_base, modo_acceso, estado_legal, url_terminos, rate_limit, activa	Registro de fuentes (RF-FUE)
`search_profiles`	source_id, criterios (JSON), versión, activo	Qué buscar en cada fuente
`runs`	source_id, versión de perfil, inicio, fin, estado (ok, parcial, bloqueada, cambio de estructura), páginas, ítems, errores, costo	Salud de la ingesta
`raw_snapshots`	run_id, url, hash, ruta del archivo, fecha	Re-extraer sin volver a la fuente y armar fixtures de test
`projects`	source_id, external_id, url, título, descripción, skills, presupuesto mín/máx, moneda, tipo (fijo u hora), publicado, propuestas, país del	Proyecto normalizado

	publicado, propuestas, país del cliente, pago verificado, primera y última vez visto	
`project_versions`	project_id, snapshot_id, campos cambiados, fecha	Histórico para tendencias
`classifications`	project_id, categoría, subcategorías, idioma, relevancia, justificación, modelo, versión de prompt	Filtro (RF-CLA)
`ratings`	tipo (post o fuente), objetivo_id, puntaje, desglose por criterio, versión de rúbrica	Rating (RF-RAT)
`labels`	project_id, campo, valor esperado, autor, fecha	Golden set y correcciones humanas
`brief_analyses`	project_id, ficha (JSON), preguntas, banderas rojas	Análisis de brief
`profitability`	project_id, horas por escenario, tarifa, comisión, margen por escenario, prioridad, versión de parámetros	Rentabilidad
`market_metrics`	semana, fuente, categoría, mediana, p25, p75, volumen, competencia media	Tendencias
`agent_calls`	run_id, agente, modelo, tokens de entrada y salida, costo, latencia, resultado	Trazas y costo por agente

Dashboard de validación

En la fase 1 el dashboard es una herramienta de control de calidad, no un producto: su trabajo es responder «¿el scraping trae lo correcto y el análisis tiene sentido?». La propuesta es Streamlit leyendo de Neon, porque se escribe en Python, no requiere frontend y puede guardar etiquetas en la misma base.

Vista	Qué muestra	Pregunta que responde	Fase
Salud de ingesta	Corridas por fuente con estado, ítems, errores, alertas de cambio de estructura y duración	¿Funcionó la última corrida?	1
Auditoría de extracción	Snapshot original al lado de los campos extraídos; marcar cada campo como correcto o incorrecto	¿Se extrajo bien este proyecto?	1

	campo como correcto o incorrecto		
Bandeja de proyectos	Tabla filtrable por categoría, relevancia, rating, presupuesto y fecha; detalle con la justificación del agente	¿Qué proyectos valen la pena hoy?	1
Calidad del filtro	Matriz de confusión y precisión/recall contra el golden set, por versión de prompt	¿Mejoró o empeoró el clasificador?	1
Fuentes	Rating por fuente con su desglose y estado legal	¿Dónde conviene buscar?	1
Costos	Tokens y USD por agente, por corrida y por proyecto procesado	¿Cuánto cuesta cada corrida?	1
Rentabilidad	Margen por escenario y prioridad por proyecto	¿Dónde está el mejor margen esperado?	2
Mercado	Mediana y rango de presupuestos por categoría y semana, volumen y competencia	¿Cómo se mueven los precios?	2

- **RF-DSH-01** — Cuando una persona marca un campo como incorrecto en la auditoría, el sistema debe guardar el valor correcto en `labels`.
- **RF-DSH-02** — El dashboard debe mostrar para cada número la corrida y la versión de prompt o rúbrica que lo produjo.
- **RF-DSH-03** — El dashboard debe ser de solo lectura sobre los datos de ingesta; solo escribe etiquetas y correcciones.

Requerimientos no funcionales

El punto bloqueante es legal: los términos de Workana prohíben el scraping, así que la fuente inicial necesita una decisión antes de escribir el conector.

Términos de uso de las fuentes

Los Términos y Condiciones de Workana prohíben lanzar programas o scripts para scrapear, indexar o hacer minería de datos del sitio, y usar robots o procesos manuales o automáticos con ese fin. La licencia de uso es personal y no comercial, y Workana puede cerrar cuentas en cualquier momento. Además, el sitio bloquea accesos automatizados con detección de bots.

Opciones a decidir (no es asesoramiento legal; conviene validarlo con un abogado):

1. Pedir autorización escrita a Workana o un acuerdo de acceso a datos.
2. Arrancar la fase 1 con fuentes que ofrecen API oficial, por ejemplo Freelancer.com, revisando sus términos de API.

3. Usar las notificaciones de proyectos que la plataforma envía a la propia cuenta, con validación legal previa.

En todos los casos rigen RF-FUE-03 (no se ejecuta una fuente sin estado «permitido») y RF-ING-03 (no se evaden CAPTCHA ni bloqueos).

Datos personales y propiedad intelectual

- No se guardan nombres, usuarios ni contactos de quienes publican; los términos de Workana también prohíben recolectarlos.
- El contenido generado para clientes debe ser original y con fuentes citadas; lo valida el citation checker.

Costos

- Tope de costo por corrida configurable; si se supera, la corrida se detiene y queda marcada.
- Modelo chico para clasificación y rating; modelo grande solo para brief, rentabilidad y redacción.
- Clasificar con reglas antes que con LLM (RF-CLA-02) y reutilizar resultados por hash del contenido.

Observabilidad y reproducibilidad

- Cada corrida tiene un id; cada llamada de agente queda en `agent_calls` con tokens, costo, latencia y resultado.
- Prompts, rúbricas, tablas de referencia y criterios viven versionados en git.
- Los snapshots crudos permiten re-extraer y re-evaluar sin volver a la fuente.

Seguridad

- Secretos en `.env` (con `.env.example` versionado) y `DATABASE_URL` de Neon.
- Ningún agente inicia sesión ni guarda credenciales de plataformas.
- Los agentes de ingesta solo tienen las herramientas que necesitan (lectura de red y escritura en su carpeta).

Evals

Cada etapa tiene su eval antes de que la siguiente dependa de ella: no se analiza rentabilidad sobre un scraping sin medir. Los evals corren desde la consola con un comando y sus resultados quedan en `evals/results/` y en el dashboard.

Golden set

- 150 a 200 proyectos etiquetados a mano por dos personas: campos extraídos, categoría, relevancia y calidad de 1 a 5.

- Acuerdo entre las dos personas (kappa de Cohen ⩾ 0,6) antes de usar la rúbrica: si los humanos no coinciden, la rúbrica está mal definida.
- Incluir negativos difíciles (un proyecto de desarrollo que menciona SEO, un logo que menciona UX), y posts en español, portugués e inglés.
- División fija en desarrollo y test; los prompts se ajustan solo contra desarrollo.
- Los snapshots del golden set se guardan como fixtures, así el eval no depende de la fuente en vivo.

Eval por etapa

Etapa	Tipo de eval	Métrica	Umbral inicial	Cuándo corre
Extracción	Determinista sobre fixtures	Exactitud por campo, % de campos vacíos	⩾ 95%	Cada cambio de parser
Ingesta en vivo	Monitoreo	Ítems por corrida vs. promedio de 4 semanas	Alerta si cae a la mitad	Cada corrida
Deduplicación	Pares etiquetados	Precisión / recall de duplicados	⩾ 95% / ⩾ 90%	Cada cambio de regla
Clasificación	Golden set	Precisión, recall y F1 por categoría; matriz de confusión	⩾ 90% / ⩾ 85%	Cada cambio de prompt o modelo
Rating de post	Contra humanos	Spearman; acuerdo dentro de ±1 punto en escala 1–5	⩾ 0,6; ⩾ 80%	Cada cambio de rúbrica
Análisis de brief	LLM-as-judge + muestra humana	Cobertura de campos; afirmaciones que no están en el post	⩾ 4 de 5; cero inventadas	Cada cambio de prompt
Rentabilidad	Tests unitarios + backtest	Cálculos exactos; error de horas estimadas vs. reales en proyectos ganados	100% de tests; error a medir	Cada cambio y mensual
Tendencias	Consistencia	Números del resumen presentes en el resultado SQL	100%	Cada resumen
Investigador de fuentes	Revisión humana	Fuentes aceptadas; enlaces y términos válidos	A medir	Cada propuesta
Contenido SEO/GEO	Determinista	Title, meta, H1, longitudes, datos	100% de reglas	Cada página

		estructurados válidos		
Citas y originalidad	Citation checker + similitud	Afirmaciones con fuente verificada; similitud con las fuentes	100%; bajo umbral	Cada página
Voz y estilo	LLM-as-judge por pares	Preferencia frente a la versión anterior	Gana o empata	Cada cambio de guía
Web construida	Lighthouse y axe	Accesibilidad, rendimiento, SEO; enlaces rotos	⩾ 90; cero	Cada build

Regresión y jueces

1. Todo cambio de prompt, rúbrica, parser o modelo corre la suite y se compara contra la última línea base.
2. Si una métrica cae más de 2 puntos, el cambio no se adopta hasta explicarlo.
3. Un LLM-as-judge se valida primero contra etiquetas humanas (acuerdo ⩾ 80%) y usa un prompt distinto al del generador.
4. Las correcciones humanas del dashboard alimentan el golden set de la siguiente versión.

La métrica de negocio final, desde la fase 3, es la tasa de propuestas ganadas y el margen real frente al estimado.

Implementación en Claude Code

La sesión principal orquesta siguiendo el pipeline de `CLAUDE.md`; cada etapa es un subagente escrito en prosa que llama scripts de Python para todo lo determinista. Así los prompts se pueden migrar después a nodos de LangGraph sin reescribir la lógica.

Estructura del repositorio

```
├── CLAUDE.md                 # pipeline paso a paso y reglas globales
├── .claude/
│   ├── agents/               # un .md por agente: clasificador, evaluador-
calidad,
│   │                         # analista-brief, analista-rentabilidad,
analista-mercado…
│   ├── skills/               # rúbrica de calidad, tabla de horas, esquema de
proyecto,
│   │                         # un conector por fuente
│   └── commands/             # /ingest, /classify, /rate, /analyze, /market,
/eval
├── config/                   # sources.yaml, search_profiles.yaml, parámetros
de rentabilidad
```

```
de rentabilidad
├── scripts/                  # conectores, parsers, dedup, cálculos, consultas
SQL
├── evals/                    # golden/, fixtures/, results/
├── dashboard/                # app Streamlit
├── specs/                    # 001-registro-fuentes/, 002-ingesta/… (spec
EARS, plan, tasks)
└── .env.example
```

Comandos de consola

Comando	Qué hace	Quién trabaja
`/ingest <fuente>`	Trae listados, guarda snapshots, extrae, deduplica y versiona	Scripts (sin LLM)
`/classify [corrida]`	Reglas de palabras clave y clasificación de los ambiguos	Subagente clasificador
`/rate [corrida]`	Rating de posts y recálculo del rating de fuentes	Subagente evaluador de calidad
`/analyze <proyecto o top N>`	Ficha del brief, horas por escenario y margen	Analistas de brief y rentabilidad + scripts
`/market [semana]`	Métricas por SQL y resumen redactado	Script + analista de mercado
`/eval <etapa>`	Corre la suite y compara contra la línea base	Scripts + jueces

Las corridas sin intervención se lanzan en modo no interactivo, por ejemplo `claude -p "/ingest freelancer"`, desde la consola o un cron.

Decisiones de orquestación

- **Orquestación plana.** Claude Code permite que un subagente lance otros, por defecto hasta tres niveles (documentación). Aun así conviene que la sesión principal llame a cada subagente en secuencia y fijar `CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH=1`, para que cada paso quede trazado y con su costo; los subniveles del diagrama pasan a etapas hermanas o skills.
- **Pasaje por datos, no por contexto.** Cada subagente lee y escribe en la base (o en archivos de la corrida) y devuelve un resumen corto; el orquestador no acumula proyectos en su contexto.
- **Salidas con esquema.** Todo agente devuelve JSON validado contra un esquema en `skills/esquema-*`; eso hace posibles los evals y la migración.
- **Herramientas mínimas.** Cada agente declara solo las herramientas que usa; el clasificador no tiene red ni escritura de archivos.

- **SDD.** Una carpeta numerada por módulo en `specs/` (constitution, spec en EARS, plan y tasks), empezando por registro de fuentes e ingesta.

Riesgos y preguntas abiertas

El riesgo mayor no es técnico sino de acceso a los datos; el segundo es construir análisis y generación sobre una ingesta que nadie midió.

Riesgo	Impacto	Mitigación
Los términos de la fuente prohíben el scraping	Cierre de cuenta, bloqueo, exposición legal	Decidir la fuente antes de construir; priorizar API oficial; estado legal por fuente
La fuente cambia su HTML	Datos rotos sin que nadie lo note	Alerta de cambio de estructura, fixtures y monitoreo de volumen
El análisis de brief inventa requisitos	Cotizaciones y decisiones equivocadas	Cada dato de la ficha cita el texto del post; eval de cero afirmaciones inventadas
Horas estimadas sin datos reales	Márgenes irreales	Tabla de referencia del equipo y backtest con proyectos ganados
El costo de tokens crece con el volumen	Corridas caras	Reglas antes que LLM, modelo chico para clasificar, tope por corrida
Rúbrica de calidad subjetiva	Rating que no refleja el criterio del equipo	Doble etiquetado con kappa y revisión mensual
Saltar a contenido y web antes de validar	Producir sobre oportunidades mal elegidas	Criterios de salida por fase

Supuestos

- El equipo ofrece servicios de UX/UI, SEO y desarrollo web.
- Existen una tarifa interna por hora y comisiones conocidas por plataforma.
- Hay dos personas disponibles para etiquetar el golden set.

Preguntas abiertas

- [ ] ¿Con qué fuente arranca la fase 1, dados los términos de Workana: autorización, API oficial de otra plataforma o notificaciones de la cuenta?
- [ ] ¿Qué es el «gold-standard de páginas o dominios» del diagrama: fuentes de referencia, posts ejemplares para calibrar el rating o sitios modelo para el contenido?
- [ ] Cuando el pedido dice «copyright», ¿se refiere a copywriting, a derechos de autor o a ambos? El documento asume ambos.